

Stack ADT, Operations & Array Implementation

Q. Define Stack as an Abstract Data Type (ADT). Explain its primary operations and demonstrate how a Stack can be implemented using an Array with suitable code snippets.

****Definition to Remember****

A ****Stack**** is a linear data structure in which elements can be removed or added only from one end, i.e., the top. It follows the Last In First Out (LIFO) principle. An empty stack is represented by top = -1.

1. Stack as an ADT

Operation	Description	Condition
-----------	-------------	-----------

| :--- | :-

2. Stack Diagram

Initial (Empty): After push(10, 20, 30):

top = -1

3. Array Implementation in C

```
#include <stdio.h>
```

```
#define
```

```
int stack[MAX];
```

```
int top =
```

```
void push(int value) {
```

```
if (top =
```

```
int pop() {
```

```
if (top =
```

```
int peek() {
```

```
if (top ==
```

```
int isEmpty() { return top == -1; }
```

4. Advantages and Limitations

| | Array Implementation |

| :--- | :-

*****Must-Write Points (for 10 marks)*****

1. Stack

*****Quick Recall*****

`Stack -

Complete Executable C Code: Array Implementation of Stack ADT

```
#include <stdio.h>
```

```
#include
```

```
#define MAX 5
```

```
struct Stack {
```

```
int item
```

```
void initStack(struct Stack *s) {
```

```
s->top =
```

```
int isFull(struct Stack *s) {
```

```
return s
```

```
int isEmpty(struct Stack *s) {
```

```
return s
```

```
void push(struct Stack *s, int val) {
```

```
if (isFull)
```

```
int pop(struct Stack *s) {
```

```
if (isEmpty)
```

```
int peek(struct Stack *s) {
```

```
if (isEmpty)
```

```
void display(struct Stack *s) {
```

```
if (isEmpty)
```

```
int main() {
```

```
struct S
```

```
push(&s, 10);
```

```
push(&
```

```
printf("Popped Value: %d\n", pop(&s));
```

```
display
```

```
printf("Top Element (Peek): %d\n", peek(&s));
```

```
return C
```